Hardware Port And Wiring Reference

Project: Smart Classroom Energy-Saving, Safety & Asset Monitoring System
Repository: https://github.com/3351666087/desktop-tutorial

This document explains the hardware interfaces used in the final demonstration. It is written for tutor review, so it focuses on the actual implemented wiring and why each connection is needed.

1. Hardware Roles

Device	Role In The System
Arduino UNO R3	Edge control node. It reads most sensors and directly controls the lamp, buzzer, relay fan, fan PWM and status lights.
ESP32	IoT gateway. It connects to Wi-Fi/MQTT, forwards telemetry and commands, and reads the LM35 through its ADC route.
Laptop	Local service layer. It runs MQTT broker, web dashboard, WebSocket live updates and AI backend.
12 V fan and adapter	Physical ventilation actuator, switched by relay and controlled by PWM.

The UNO and ESP32 share ground for UART and signal reference. The UNO and ESP32 are powered independently by USB or adapters; the UNO 5 V line is not used to power the ESP32.

2. Arduino UNO Pin Map

Function	UNO Pin	Direction	Connected Module / Signal
PIR motion detection	D2	input	PIR OUT
Fan tachometer feedback	D3	interrupt input	4-wire fan yellow tach/FG
Emergency test button	D4	input pull-up	tactile button to GND
Classroom lamp simulation	D5	output	LED through 220 ohm resistor
Fan relay control	D6	output	relay IN
Active buzzer	D7	output	active buzzer signal pin
Flame digital output	D8	input	flame sensor DO
Fan PWM control	D9	PWM output	4-wire fan blue PWM
Status light red	D10	output	traffic light R
Status light yellow	D11	output	traffic light Y
Status light green	D12	output	traffic light G
Ambient light sensor	A0	analog input	TEMT6000 SIG

Function	UNO Pin	Direction	Connected Module / Signal
Gas sensor	A1	analog input	gas sensor AO
Legacy temperature input	A2	analog input	reserved / earlier LM35 route
Flame analog output	A3	analog input	flame sensor AO
UART RX from ESP32	A4	SoftwareSerial RX	ESP32 GPIO17 TX, optional command return
UART TX to ESP32	A5	SoftwareSerial TX	ESP32 GPIO16 RX through voltage divider

3. ESP32 Pin Map

Function	ESP32 Pin	Direction	Connected Signal
UART RX2 from UNO	GPIO16	input	UNO A5 through resistor divider
UART TX2 to UNO	GPIO17	output	UNO A4, optional return command path
LM35 temperature ADC	GPIO34	analog input	LM35 OUT
Dummy ADC reference	GPIO35	analog input	floating/reference compensation route
UART TX guard	GPIO25	input guard	prevents accidental conflict on unused UART TX route

GPIO34 and GPIO35 are input-only ADC pins, which makes them suitable for sensor reading and compensation. The LM35 is powered from the ESP32 5 V route in the final build because this reduced the abnormal high reading observed when it was powered from the UNO side.

4. Sensor Connections

Sensor	Power	Signal	Purpose
PIR motion sensor	UNO 5 V / GND	OUT to UNO D2	Detects classroom occupancy or movement.
TEMT6000 light sensor	UNO 5 V / GND	SIG to UNO A0	Measures ambient brightness for lighting decisions.
Gas sensor	UNO 5 V / GND	AO to UNO A1	Provides analog gas danger value. DO is not used.
Flame sensor	UNO 5 V / GND	DO to D8, AO to A3	Provides both digital flame detection and analog intensity.
Emergency button	UNO internal pull-up	D4 to GND when pressed	Safe way to demonstrate alarm mode without real fire/smoke.
LM35 temperature sensor	ESP32 5 V / GND	OUT to GPIO34	Measures temperature through ESP32 ADC path.

5. Actuator Connections

Actuator	Control Signal	Power / Load Wiring	Notes
Classroom LED	UNO D5	D5 -> 220 ohm resistor -> LED -> GND	Simulates classroom light.
Active buzzer	UNO D7	UNO 5 V / GND	Used for short confirmation and safety alarm pattern.
Traffic light module	UNO D10/D11/D12	R/Y/G to UNO pins, GND to UNO GND	Shows green/yellow/red system state.
Relay module	UNO D6	control side uses UNO 5 V/GND	Relay logic is configurable through `RELAY_ON` and `RELAY_OFF`.
12 V fan	relay + UNO D9 PWM + UNO D3 tach	12 V adapter positive through relay COM/NO to fan red, fan black to 12 V negative	Fan yellow tach goes to D3; fan blue PWM goes to D9.

The fan power path and logic signal path must share ground. The 12 V adapter negative is connected to the UNO/ESP32 ground reference so that tach and PWM signals have a valid reference.

6. UART Connection Between UNO And ESP32

Path	Wiring	Reason
UNO telemetry to ESP32	UNO A5 TX -> resistor divider -> ESP32 GPIO16 RX2	UNO output is 5 V logic, ESP32 RX is 3.3 V tolerant, so the divider protects the ESP32.
ESP32 command to UNO	ESP32 GPIO17 TX2 -> UNO A4 RX	ESP32 3.3 V TX is normally readable by UNO as HIGH.
Ground	UNO GND -> ESP32 GND	Required for UART, PWM and ADC signal reference.

The UART protocol uses 9600 baud and sends one `SC2` telemetry line per second. Commands from the dashboard are sent back over MQTT -> ESP32 -> UART -> UNO.

7. Relay And Fan Safety Notes

- The relay contact side switches the 12 V fan supply, not the UNO 5 V supply.
- `RELAY_ON` and `RELAY_OFF` are defined as constants so the code can support active-high or active-low relay modules.
- The fan can be forced on during `SAFETY_ALARM`.
- Tach feedback is interrupt-based and non-blocking; if tach is missing, RPM can be zero without stopping the system.
- PWM can be controlled by automatic temperature logic or by manual/voice commands.

8. Current Build Constraints

- No capacitors are used in the current assembly.
- `credentials.h` is local-only and ignored by Git because it contains Wi-Fi and broker IP values.

- The laptop IP may change when the Wi-Fi network changes. If the dashboard shows `waiting`, update the ESP32 MQTT host in `credentials.h` and reflash the ESP32.
- Safety and basic automatic behavior remain on the UNO, so the local edge controller is not dependent on Qwen or the web dashboard.

9. Quick Onsite Validation

Check	Expected Result
Dashboard state	`online=true`, recent telemetry age below 2 seconds
PIR movement	occupancy changes on dashboard
Cover light sensor	light value changes and lamp logic reacts
Press emergency button	red status light, buzzer alarm pattern, fan forced on
Manual fan command	fan state and PWM change, ACK appears on dashboard
Voice command	Qwen task appears, MQTT command is sent, physical actuator changes